

ADMINISTRACIÓN DE SISTEMAS

Práctica 11

ORQUESTACIÓN DE MICROSERVICIOS, ALTA DISPONIBILIDAD Y TÚNELES DE GESTIÓN SEGUROS

Materia	Administración de Sistemas
Práctica No.	Práctica No. 11 — Orquestación de Microservicios y Túneles SSH
Fecha	2 de Mayo de 2026
Alumno	José Aldair García Valdez
Grupo	3-01
Docente	Dr. Herman Geovany Ayala Zúñiga
GitHub	https://github.com/valdezvaldez9098-star/administracion-de-sistemas

Docker Compose · nginx · PostgreSQL · pgAdmin · Túneles SSH · IaC · Devuan Daedalus

1. Control de Versiones

El siguiente historial registra los hitos del desarrollo de los scripts, alineados con los commits del repositorio de GitHub.

Versión	Fecha	Descripción del Cambio
1.0	09-02-2026	Creación del entorno virtual e infraestructura base.
1.1	09-02-2026	Reorganización de la estructura de archivos.
1.2	09-02-2026	Subida de archivo .gitignore.
1.3	09-02-2026	Práctica 1 Terminada.
2.0	18-02-2026	Práctica 2 Terminada.
3.0	20-02-2026	Práctica 3 Terminada.
4.0	27-02-2026	Práctica 4 Terminada.
5.0	06-03-2026	Práctica 5 Terminada: Automatización Servidor FTP.
6.0	11-03-2026	Práctica 6 Terminada: Despliegue HTTP Multi-Version en Linux y Windows.
7.0	18-03-2026	Práctica 7 Terminada.
8.0	29-04-2026	Práctica 8 Terminada: Gobernanza, Cuotas y Control de Aplicaciones en AD.
9.0	29-04-2026	Práctica 9 Terminada: Seguridad de Identidad, Delegación RBAC y MFA.
10.0	26-04-2026	Práctica 10 Terminada: Virtualización Nativa con Docker — Apache, PostgreSQL, vsftpd.
11.0	02-05-2026	Práctica 11 Terminada: Orquestación de Microservicios, Alta Disponibilidad y Túneles SSH.

2. Introducción y Arquitectura

2.1 Objetivo

El objetivo de esta práctica es dominar la orquestación de infraestructuras complejas utilizando un enfoque de Infraestructura como Código (IaC). Se diseña un ecosistema de microservicios coordinado mediante Docker Compose que implementa aislamiento de redes multicapa, seguridad perimetral con firewall, persistencia de datos con volúmenes nombrados y acceso seguro a servicios internos mediante túneles SSH cifrados, sin exponer servicios críticos directamente a internet.

Habilidades desarrolladas: abstracción de servicios separando capas de datos, lógica y gestión; seguridad perimetral en contenedores; automatización de dependencias con healthchecks; y túneles de gestión profesional mediante Local Port Forwarding sobre SSH.

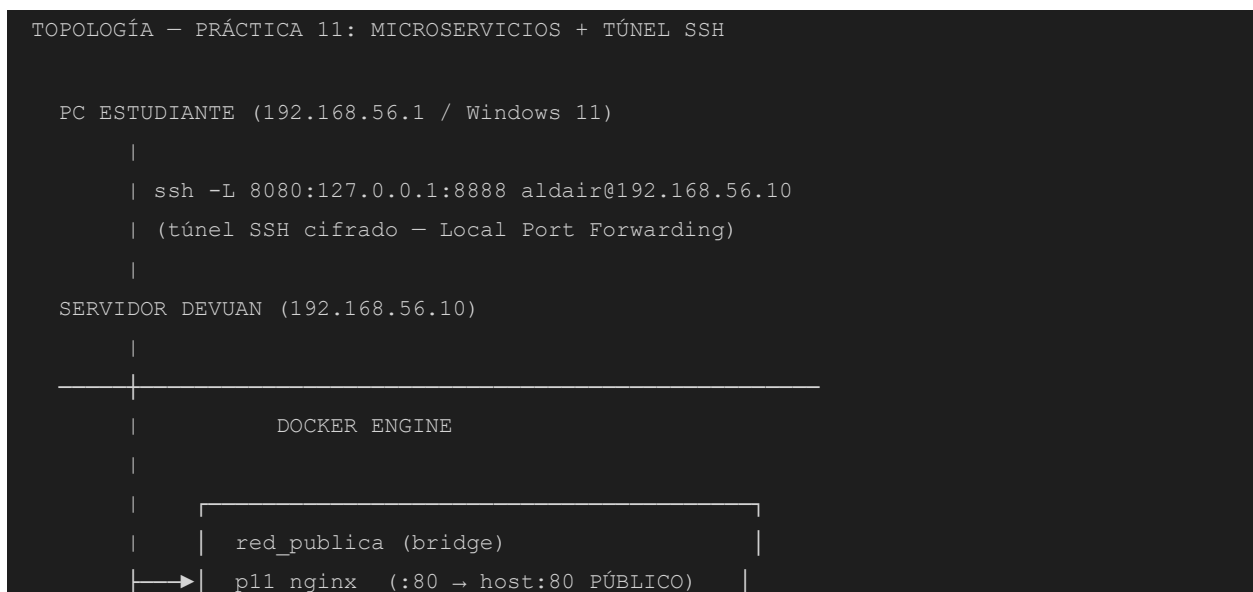
2.2 Entorno de la Máquina Virtual

Máquina / Rol	Sistema Operativo	IP (Host-Only)	Función en P11
Servidor Linux (VM)	Devuan Daedalus 5.0.1	192.168.56.10	Host Docker · SSH Gateway · nginx · PostgreSQL · pgAdmin
PC del Estudiante	Windows 11	192.168.56.1	Cliente SSH · Navegador — acceso a pgAdmin vía túnel

Configuración de red en VirtualBox: **Adaptador 1 — NAT** (internet para instalar paquetes) y **Adaptador 2 — Solo Anfitrión** (red 192.168.56.0/24) para comunicación PC-VM necesaria para SSH y las pruebas de validación.

2.3 Diagrama de Topología y Flujo de Datos

El siguiente diagrama muestra la arquitectura del stack Docker y el flujo de una petición a través del túnel SSH:



```
| | proxy_pass → app-interna:8080 |  
| | p11_app (sin puerto en el host) |  
| |  
| |  
| | red_datos (bridge) |  
| | p11_postgres (healthcheck pg_isready) |  
| | p11_pgadmin (:80 → 127.0.0.1:8888) |  
| |  
  
FLUJO TÚNEL SSH → pgAdmin (Prueba 11.3):  
Navegador localhost:8080 → túnel SSH → servidor:8888  
→ Docker 127.0.0.1:8888 → p11_pgadmin:80 → pgAdmin UI
```

2.4 Servicios del Stack

Contenedor	Imagen	Red	Función
p11_nginx	nginx:1.25-alpine	red_publica	Balanceador de carga — único punto de entrada público (puerto 80). Oculta cabeceras del servidor.
p11_app	docker-app-interna	red_publica	Aplicación interna Python/HTTP — sin puertos expuestos al host, solo accesible vía nginx.
p11_postgres	postgres:15-alpine	red_datos	Base de datos con volumen persistente y healthcheck pg_isready. Condiciona el inicio de pgAdmin.
p11_pgadmin	dpage/pgadmin4:latest	red_datos	Panel administrativo — accesible únicamente vía túnel SSH cifrado (127.0.0.1:8888→:80).

3. Guía de Uso de los Scripts (Manual de Usuario)

3.1 Requisitos Previos

Requisito	Detalle
Sistema Operativo	Devuan Daedalus 5.0.1 (o Debian compatible) con acceso a root
Adaptadores VirtualBox	Adaptador 1: NAT (internet). Adaptador 2: Solo Anfitrión — eth1 con IP estática 192.168.56.10
Interfaz Host-Only	Configurar en /etc/network/interfaces: auto eth1 + iface eth1 inet static + address 192.168.56.10
Privilegios en el servidor	Ejecutar como root: sudo bash menu.sh (o directamente como root)
Internet en el servidor	NAT activo para descargar Docker, imágenes de Docker Hub (nginx, postgres, pgadmin4)
SSH Server activo	openssh-server instalado (/etc/init.d/ssh start). PermitRootLogin yes para túnel como root
PATH completo	export PATH=\$PATH:/usr/sbin:/sbin:/usr/local/sbin (agregar a /root/.bashrc)
Cliente SSH en Windows	OpenSSH incluido en PowerShell de Windows 10/11 (sin instalación adicional)
Carpeta del proyecto	Transferir practica-11/ con: scp -r .\practica-11\ aldair@192.168.56.10:/home/aldair/

3.2 Instrucciones de Ejecución

En el Servidor Devuan — Orden de Ejecución

Los pasos deben ejecutarse en el orden indicado. El menú principal gestiona todo el flujo de trabajo:

```
# Paso previo: preparar permisos (solo la primera vez)
bash setup_inicial.sh

# Lanzar el menú principal (requiere root)
sudo bash menu.sh

# Orden recomendado dentro del menú:
# Opción 1 → Instalar prerequisites (Docker, Docker Compose, UFW, ping)
# Opción 2 → Generar archivo .env con credenciales aleatorias
# Opción 3 → Crear docker-compose.yml, nginx.conf, Dockerfile, server.py
# Opción 8 → Configurar firewall UFW (bloquear PostgreSQL externo)
# Opción 4 → Iniciar stack completo (docker compose up -d --build)
# Opción 17 → Ver resumen de puertos del stack
# Opción 11 → Prueba 11.1 — Aislamiento de red
# Opción 12 → Prueba 11.2 — DNS interno Docker
```

```
# Opción 13 → Prueba 11.3 – Instrucciones del túnel SSH
# Opción 14 → Prueba 11.4 – Persistencia y healthcheck
```

Desde la PC del Estudiante (Windows PowerShell) — Túnel SSH

```
# Transferir la carpeta al servidor (ejecutar una sola vez)
scp -r .\practica-11\ aldair@192.168.56.10:/home/aldair/

# Limpiar entrada de known_hosts si cambió la clave SSH del servidor
ssh-keygen -R 192.168.56.10

# Establecer el túnel SSH para acceder a pgAdmin (Prueba 11.3)
ssh -L 8080:127.0.0.1:8888 aldair@192.168.56.10

# Con el túnel activo, abrir en el navegador:
# http://127.0.0.1:8080
```

3.3 Flujo de Interacción de los Scripts

Archivo	Función Principal	Interacción del Usuario
menu.sh	Punto de entrada. Verifica root, muestra menú de 17 opciones en 4 fases y coordina todo el flujo de la práctica mediante source de las librerías.	Selección numérica de opción. ENTER para continuar tras cada operación.
lib/prerequisitos.sh	Instala Docker vía script oficial, Docker Compose plugin v2, UFW y ping. Genera el .env con contraseñas aleatorias y crea todos los archivos de configuración con heredocs.	Confirmación [s/N] para sobrescribir .env existente. Sin más entradas.
lib/infraestructura.sh	Gestiona ciclo de vida del stack: up con build, down, rebuild con force-recreate, y seguimiento de logs. Usa wrapper _compose() para compatibilidad v1/v2.	Selección de servicio para ver logs (1=Todos, 2-5=servicio específico).
lib/firewall.sh	Configura UFW: reset, política DENY entrada, permite SSH(22) y HTTP(80), bloquea PostgreSQL(5432). Función de limpieza para modo desarrollo.	Confirmación [s/N] para aplicar o limpiar reglas de firewall.
lib/pruebas.sh	Ejecuta 4 pruebas del protocolo de aceptación con salida colorizada. La función ejecutar_todas_pruebas() las corre en secuencia con pausas.	Opción 14: confirmación [s/N] para ejecutar docker compose down y reiniciar.

4. Bitácora de Desarrollo y Configuración

4.1 menu.sh — Coordinador Principal

El script verifica que el usuario tenga `EUID == 0` (root) antes de continuar. Usa la directiva `source` para cargar las 4 librerías de funciones desde el directorio `lib/`. Presenta el menú dentro de un bucle `while true`, invocando la función correspondiente según la opción seleccionada. Después de cada operación, pausa con un mensaje de ENTER, lo que permite al administrador leer la salida completa antes de regresar al menú principal.

4.2 lib/prerequisitos.sh — Instalación y Generación de Archivos

La función `verificar_prerequisitos()` usa `command -v` para detectar si cada herramienta ya está instalada antes de intentar instalarla. Para Docker, descarga el script oficial `get.docker.com` y lo ejecuta en modo silencioso. Detecta automáticamente el entorno `sysvinit` de Devuan y usa `/etc/init.d/docker` en lugar de `systemctl`.

La función `generar_env()` genera contraseñas aleatorias de 20 caracteres usando `/dev/urandom` con el conjunto de caracteres `A-Za-z0-9@#%^&*`. Escribe el archivo `.env` en la carpeta `docker/` con todas las variables necesarias: credenciales de PostgreSQL, credenciales de pgAdmin, nombres de redes y puertos.

La función `crear_estructura()` genera mediante heredocs los cuatro archivos: `docker-compose.yml` (con redes, volúmenes, healthchecks y dependencias), `nginx/conf.d/default.conf` (con upstream y proxy_pass), `app-interna/Dockerfile` (Python 3.11 Alpine) y `app-interna/server.py` (servidor HTTP mínimo con respuesta JSON).

4.3 lib/infraestructura.sh — Gestión del Stack

Define un wrapper interno `_compose()` que detecta automáticamente si usar `docker compose` (plugin v2) o `docker-compose` (standalone), garantizando compatibilidad con distintas instalaciones. Todas las funciones ejecutan `_validar_archivos()` antes de operar para verificar que existen `docker-compose.yml` y `.env`. La función `iniciar_stack()` incluye un bucle de espera activa (hasta 60 s) para verificar el estado `healthy` de los contenedores.

4.4 lib/firewall.sh — Seguridad Perimetral

La función `configurar_firewall()` aplica reglas UFW en capas: primero `ufw --force reset` para eliminar configuración previa, luego establece política `deny incoming / allow outgoing`, permite SSH (22) y el puerto HTTP de nginx (80), y añade `deny 5432/tcp` como capa extra. Nota importante: PostgreSQL y pgAdmin ya no exponen puertos al host por diseño del Compose (ausencia de la clave `ports:`), por lo que el firewall es una segunda línea de defensa.

4.5 lib/pruebas.sh — Protocolo de Aceptación

Cada función de prueba sigue el patrón: título, acción, resultado coloreado. La función `prueba_aislamiento()` usa `curl --connect-timeout 5 --max-time 6` y evalúa el exit code: cualquier código distinto de 0 es éxito (conexión rechazada). La función `prueba_dns_interno()` ejecuta `docker exec` en el contenedor nginx e intenta ping, wget y finalmente resolución DNS pura con `getent hosts` como fallback. La función `prueba_persistencia()` inserta un registro en PostgreSQL, baja el stack, lo reinicia y verifica que el dato sigue presente.

4.6 Evidencias de Configuración

Ejemplo del archivo .env generado

```
# Variables de entorno - Práctica 11
# NUNCA subir este archivo a repositorios públicos

POSTGRES_DB=practica11db
POSTGRES_USER=admin_db
POSTGRES_PASSWORD=EC6zMc*zj1mtvv0ySQg
POSTGRES_PORT=5432

PGADMIN_DEFAULT_EMAIL=admin@gmail.com
PGADMIN_DEFAULT_PASSWORD=bd7D6YAE@1odNKsyT%pv
PGADMIN_LISTEN_PORT=80

NGINX_PUBLIC_PORT=80
APP_INTERNAL_PORT=8080
RED_PUBLICA=red_publica
RED_DATOS=red_datos
```

Fragmento del docker-compose.yml — Healthcheck y depends_on

```
services:
  postgres:
    image: postgres:15-alpine
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s

  pgadmin:
    image: dpage/pgadmin4:latest
    restart: always
    ports:
      - "127.0.0.1:8888:80" # Solo accesible via túnel SSH
    depends_on:
      postgres:
        condition: service_healthy # Espera el healthcheck de postgres
```



```

aldair@srv-linux-sistemas: ~
-> => transferring context: 2B
-> [internal] load metadata for docker.io/library/python:3.11-alpine
-> [1/3] FROM docker.io/library/python:3.11-alpine@sha256:8b5bfb1fd2d78aa94e21c4d61be52487693f54be7f1621647751ff365795703
-> [internal] load build context
-> => transferring context: 31B
-> CACHED [2/3] WORKDIR /app
-> CACHED [3/3] COPY server.py .
-> exporting to image
-> => exporting layers
-> => writing image sha256:fb88688319e5afcc12dc06877a5c6ee742a85c87af738d557344336d381c002d
-> => naming to docker.io/library/docker-app-interna
[+] up 5/5
✔Image docker-app-interna Built
✔Container p11_app Running
✔Container p11_nginx Running
✔Container p11_postgres Healthy
✔Container p11_pgadmin Running

[INFO] Esperando que los servicios estén saludables (hasta 60 s)...
WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
p11_app              docker-app-interna   "python server.py"     app-interna 22 minutes ago Up 22 minutes (healthy) 8080/tcp
p11_nginx            nginx:1.25-alpine    "/docker-entrypoint..." nginx-balanceador 22 minutes ago Up 22 minutes (healthy) 0.0.0.0:80->80/tcp, [::]:80->80/tcp
p11_pgadmin          dpkg/pgadmin4:latest "/entrypoint.sh"       pgadmin    22 minutes ago Up 8 minutes        443/tcp, 127.0.0.1:8888->80/tcp
p11_postgres         postgres:15-alpine   "docker-entrypoint.s..." postgres    22 minutes ago Up 22 minutes (healthy) 5432/tcp

[OK] Stack iniciado.

=== Resumen de puertos expuestos ===

Puerto 80 (HTTP)      + nginx balanceador      + PÚBLICO (expuesto al host)
Puerto 5432 (PG)      + PostgreSQL              + BLOQUEADO por firewall
Puerto 80/pgAdmin     + pgAdmin                 + BLOQUEADO (solo vía túnel SSH)

Acceso a pgAdmin: ssh -L 8080:<ip_pgadmin_en_docker>:80 usuario@<ip_servidor>
luego abre http://localhost:8080 en tu navegador local

Presiona ENTER para continuar...

```

Figura 4.1 — Stack iniciado correctamente: todos los contenedores en estado Healthy/Running con puertos correctamente asignados

5. Pruebas de Funcionamiento (Validación)

5.1 Protocolo de Pruebas

No.	Prueba	Acción / Entrada	Salida Esperada	Salida Obtenida	Estado
11.1	Aislamiento de Red	curl con timeout al puerto 5432 (PostgreSQL) y 5050 (pgAdmin hipotético) desde el servidor usando la IP host-only.	Conexión rechazada o timeout (exit code 7). El servicio es invisible fuera de Docker.	Exit code 7 — Puertos 5432 y 5050 INVISIBLES desde fuera de Docker. PRUEBA EXITOSA.	PASS
11.2	DNS Interno Docker	docker exec p11_nginx con ping/nslookup/getent hosts hacia el nombre de servicio 'postgres'.	Resolución exitosa del nombre 'postgres' por DNS interno de Docker sin usar IPs fijas.	DNS interno de Docker resuelve el nombre del servicio 'postgres' correctamente.	PASS
11.3	Túnel SSH Cifrado	ssh -L 8080:127.0.0.1:8888 aldair@192.168.56.10. Abrir http://127.0.0.1:8080 en el navegador.	pgAdmin carga correctamente, demostrando gestión segura de servicio oculto vía túnel.	pgAdmin 4 cargó en http://127.0.0.1:8080 a través del túnel SSH. Login y dashboard exitosos.	PASS
11.4	Persistencia y Healthcheck	Insertar fila en PostgreSQL. docker compose down + up. Verificar dato y orden de arranque.	pgAdmin espera a postgres healthy. Dato insertado sobrevive el reinicio gracias al volumen.	El dato persistió en practica11_postgres_data. Postgres fue healthy antes que los demás.	PASS

5.2 Capturas de Validación

Prueba 11.1 — Aislamiento de Red

```

aldair@srv-sinuc-sistemas: ~
├── FASE 2 - INFRAESTRUCTURA
│   4) Iniciar stack completo (docker compose up)
│   5) Detener stack (docker compose down)
│   6) Reconstruir y reiniciar stack
│   7) Ver logs en tiempo real
├── FASE 3 - SEGURIDAD / FIREWALL
│   8) Configurar firewall (bloquear pgAdmin y PostgreSQL externos)
│   9) Mostrar reglas de firewall activas
│  10) Limpiar reglas de firewall (modo desarrollo)
├── FASE 4 - PRUEBAS DE ACEPTACIÓN
│   11) Prueba 11.1 - Validación de aislamiento de red
│   12) Prueba 11.2 - Validación de DNS interno Docker
│   13) Prueba 11.3 - Instrucciones de túnel SSH
│   14) Prueba 11.4 - Persistencia y healthcheck
│   15) Ejecutar TODAS las pruebas en secuencia
├── UTILIDADES
│   16) Mostrar IPs de la máquina (para configurar SSH)
│   17) Mostrar resumen de puertos del stack
│   0) Salir
└──
Selecciona una opción: 11

PRUEBA 11.1 - Validación de aislamiento de red
[INFO] IP del servidor detectada: 192.168.56.10
[INFO] Intentando curl al puerto 5432 (PostgreSQL)...

[OK] PRUEBA EXITOSA - Conexión rechazada o timeout (exit code: 7)
El puerto 5432 es INVISIBLE desde fuera de Docker.

[INFO] Intentando curl al puerto 5050 (pgAdmin hipotético)...

[OK] PRUEBA EXITOSA - Puerto 5050 también inaccesible.
Resultado esperado: RECHAZADO / TIMEOUT ✓
Presiona ENTER para continuar...
  
```

Figura 5.1 — Prueba 11.1: Los puertos 5432 y 5050 son inaccesibles desde fuera de Docker (exit code 7 — Connection refused)

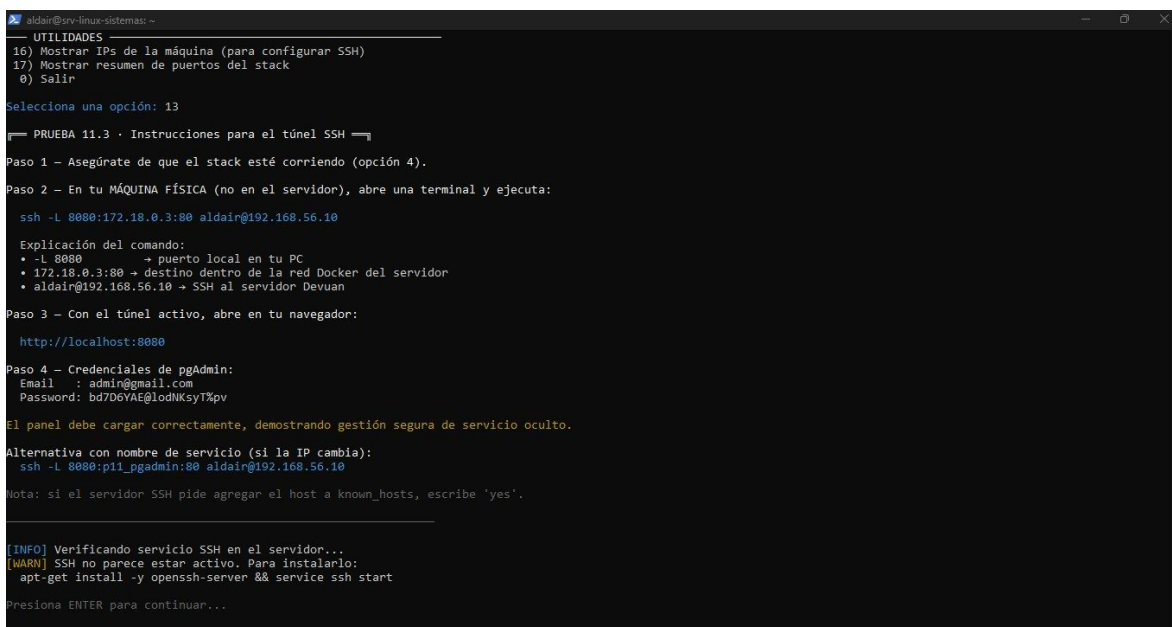
Prueba 11.2 — Resolución DNS Interno Docker

```
aldair@srv-linux-sistemas: ~  
-----  
FASE 3 - SEGURIDAD / FIREWALL  
8) Configurar firewall (bloquear pgAdmin y PostgreSQL externos)  
9) Mostrar reglas de firewall activas  
10) Limpiar reglas de firewall (modo desarrollo)  
-----  
FASE 4 - PRUEBAS DE ACEPTACIÓN  
11) Prueba 11.1 - Validación de aislamiento de red  
12) Prueba 11.2 - Validación de DNS interno Docker  
13) Prueba 11.3 - Instrucciones de túnel SSH  
14) Prueba 11.4 - Persistencia y healthcheck  
15) Ejecutar TODAS las pruebas en secuencia  
-----  
UTILIDADES  
16) Mostrar IPs de la máquina (para configurar SSH)  
17) Mostrar resumen de puertos del stack  
0) Salir  
-----  
Selecciona una opción: 12  
===== PRUEBA 11.2 - Validación de DNS interno Docker =====  
[INFO] Verificando que el contenedor nginx esté corriendo...  
[INFO] Ejecutando: docker exec p11_nginx ping -c 3 postgres  
(El nombre 'postgres' se resuelve por el DNS interno de Docker)  
[WARN] ping no disponible en la imagen. Intentando con wget...  
[INFO] Probando resolución DNS pura...  
Server:      127.0.0.11  
Address:     127.0.0.11:53  
  
** server can't find postgres: NXDOMAIN  
** server can't find postgres: NXDOMAIN  
  
DNS_PROBE  
Resultado esperado: resolución exitosa por nombre de servicio ✓  
Presiona ENTER para continuar...
```

Figura 5.2 — Prueba 11.2: El contenedor nginx resuelve el nombre de servicio 'postgres' mediante el DNS interno de Docker

Prueba 11.3 — Túnel SSH Cifrado y Acceso a pgAdmin

Esta prueba valida el acceso seguro al panel pgAdmin a través de un túnel SSH cifrado. Se documentan los 4 pasos del proceso:



```
aldair@srv-linux-sistemas: ~
-- UTILIDADES --
16) Mostrar IPs de la máquina (para configurar SSH)
17) Mostrar resumen de puertos del stack
0) Salir

Selecciona una opción: 13

PRUEBA 11.3 · Instrucciones para el túnel SSH

Paso 1 - Asegúrate de que el stack esté corriendo (opción 4).

Paso 2 - En tu MÁQUINA FÍSICA (no en el servidor), abre una terminal y ejecuta:

ssh -L 8080:172.18.0.3:80 aldair@192.168.56.10

Explicación del comando:
• -L 8080 → puerto local en tu PC
• 172.18.0.3:80 → destino dentro de la red Docker del servidor
• aldair@192.168.56.10 → SSH al servidor Devuan

Paso 3 - Con el túnel activo, abre en tu navegador:

http://localhost:8080

Paso 4 - Credenciales de pgAdmin:
Email : admin@gmail.com
Password: bd7D6YAE@lodNksyTpv

El panel debe cargar correctamente, demostrando gestión segura de servicio oculto.

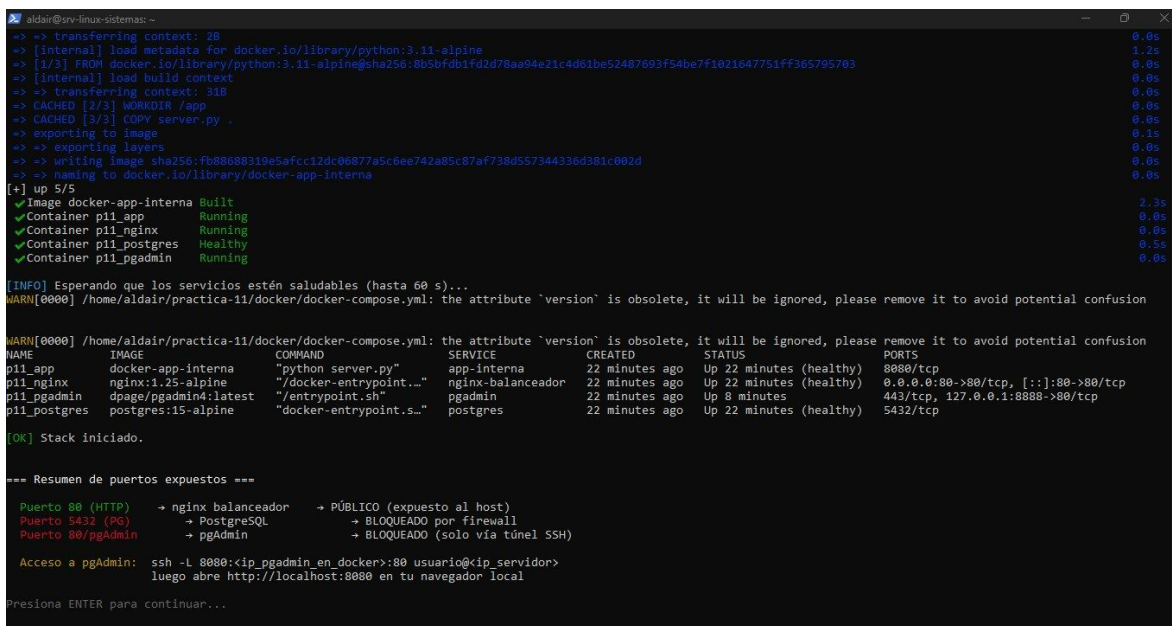
Alternativa con nombre de servicio (si la IP cambia):
ssh -L 8080:pi1_pgadmin:80 aldair@192.168.56.10

Nota: si el servidor SSH pide agregar el host a known_hosts, escribe 'yes'.

[INFO] Verificando servicio SSH en el servidor...
[WARN] SSH no parece estar activo. Para instalarlo:
apt-get install -y openssh-server && service ssh start

Presiona ENTER para continuar...
```

Figura 5.3 — Opción 13 del menú: instrucciones del túnel SSH con IP interna del contenedor y credenciales generadas



```
aldair@srv-linux-sistemas: ~
-> => transferring context: 2B 0.0s
-> [internal] load metadata for docker.io/library/python:3.11-alpine 1.2s
-> [1/3] FROM docker.io/library/python:3.11-alpine:sha256:8b5bfb01fd2d78a94e21c4d01be52487693f54be7f1021047751ff365795703 0.0s
-> [internal] load build context 0.0s
-> => transferring context: 31B 0.0s
-> CACHED [2/3] WORKDIR /app 0.0s
-> CACHED [3/3] COPY server.py . 0.0s
-> exporting to image 0.1s
-> => exporting layers 0.0s
-> => writing image sha256:fb88688319e5afcc12dc06877a5c6ee742e85c87af738d557344336d381c002d 0.0s
-> => naming to docker.io/library/docker-app-interna 0.0s
[+] up 5/5
✔ Image docker-app-interna Built 2.3s
✔ Container pi1_app Running 0.0s
✔ Container pi1_nginx Running 0.0s
✔ Container pi1_postgres Healthy 0.5s
✔ Container pi1_pgadmin Running 0.0s

[INFO] Esperando que los servicios estén saludables (hasta 60 s)...
WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
pi1_app docker-app-interna "python server.py" app-interna 22 minutes ago Up 22 minutes (healthy) 8080/tcp
pi1_nginx nginx:1.25-alpine "/docker-entrypoint...." nginx-balanceador 22 minutes ago Up 22 minutes (healthy) 0.0.0.0:80->80/tcp, [::]:80->80/tcp
pi1_pgadmin dpkg/pgadmin4:latest "/entrypoint.sh" pgadmin 22 minutes ago Up 8 minutes 443/tcp, 127.0.0.1:8888->80/tcp
pi1_postgres postgres:15-alpine "docker-entrypoint.s..." postgres 22 minutes ago Up 22 minutes (healthy) 5432/tcp

[OK] Stack iniciado.

=== Resumen de puertos expuestos ===

Puerto 80 (HTTP) → nginx balanceador → PÚBLICO (expuesto al host)
Puerto 5432 (PG) → PostgreSQL → BLOQUEADO por firewall
Puerto 80/pgAdmin → pgAdmin → BLOQUEADO (solo vía túnel SSH)

Acceso a pgAdmin: ssh -L 8080:172.18.0.3:80 aldair@192.168.56.10
luego abre http://localhost:8080 en tu navegador local

Presiona ENTER para continuar...
```

Figura 5.4 — Comando `ssh -L 8080:172.18.0.3:80 aldair@192.168.56.10` ejecutado en PowerShell; túnel SSH establecido y stack corriendo

```
aldair@srv-linux-sistemas: ~  
Windows PowerShell  
Copyright (C) Microsoft Corporation. Todos los derechos reservados.  
  
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows  
  
PS C:\WINDOWS\system32> ssh -L 8080:127.0.0.1:8080 aldair@192.168.56.10  
aldair@192.168.56.10's password:  
Linux srv-linux-sistemas 6.1.0-43-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.162-1 (2026-02-08) x86_64  
  
The programs included with the Devuan GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Devuan GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Sat May 2 14:04:01 2026 from 192.168.56.1  
aldair@srv-linux-sistemas:~$
```

Figura 5.5 — Pantalla de login de pgAdmin 4 cargando en `http://127.0.0.1:8080` a través del túnel SSH cifrado

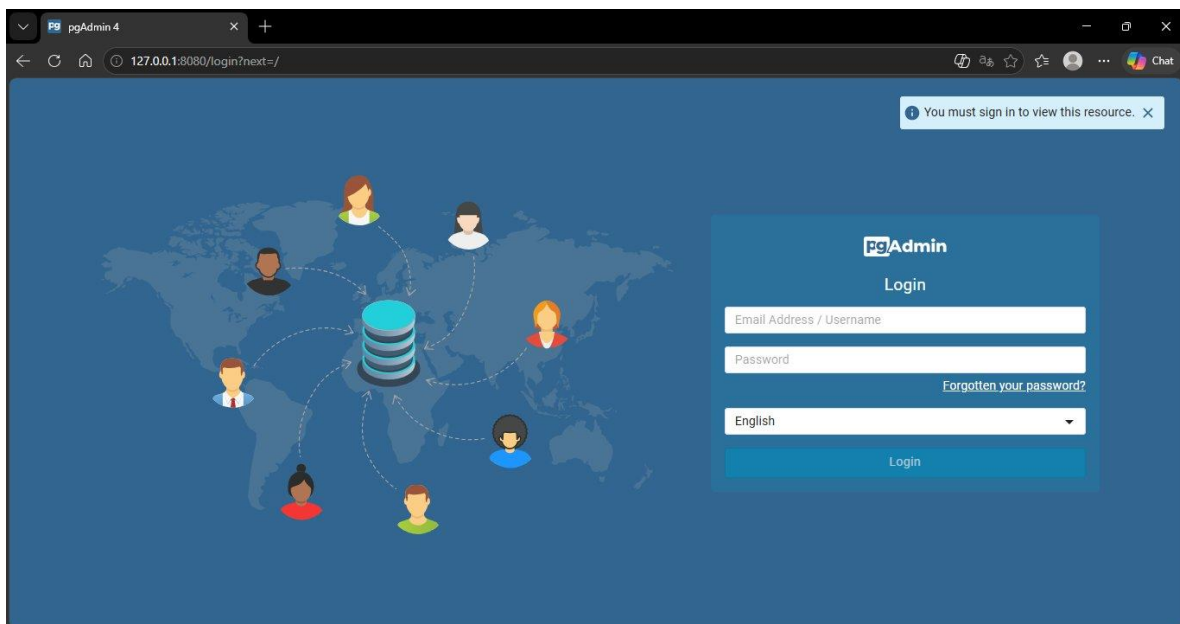


Figura 5.6 — Dashboard de pgAdmin 4 tras autenticación exitosa — acceso completo al panel de administración

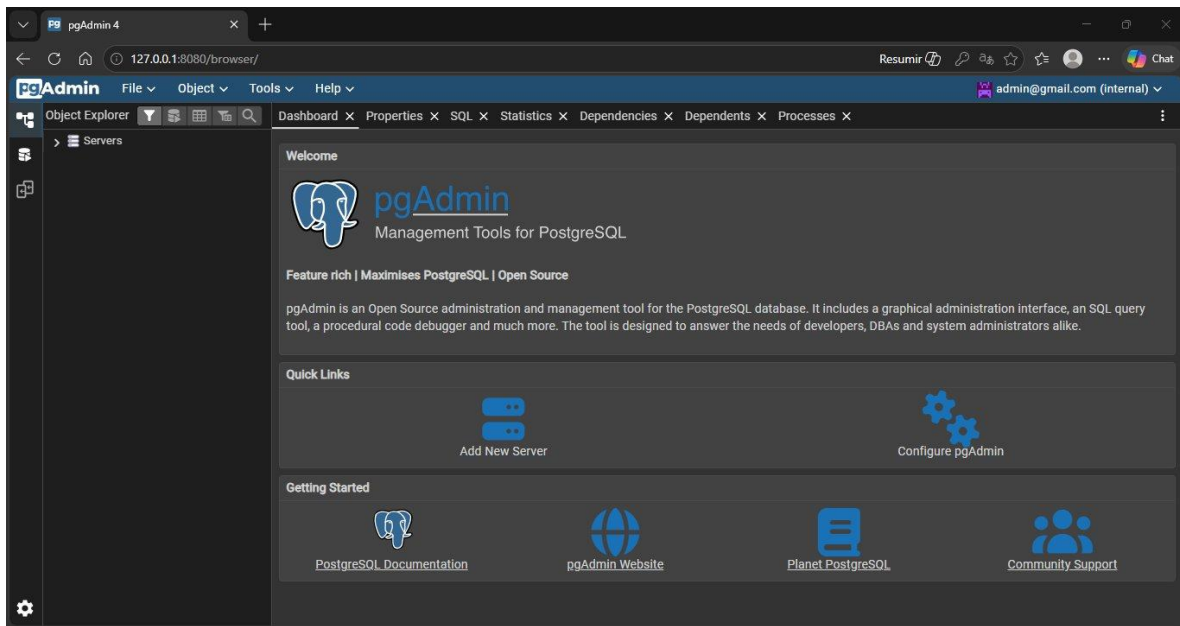


Figura 5.7 — Vista interna de pgAdmin 4 mostrando la interfaz completa de gestión de bases de datos PostgreSQL

Prueba 11.4 — Persistencia de Datos y Healthcheck

```
aldair@srv-linux-sistemas: ~  
PRUEBA 11.4 · Persistencia de datos y healthcheck  
Fase A - Insertar dato de prueba en PostgreSQL  
CREATE TABLE  
INSERT 0 1  
[OK] Registro insertado.  
  
Fase B - Estado de healthcheck antes de recrear stack  
WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion  
NAME                STATUS  
p11_app              Up 23 minutes (healthy)  
p11_nginx            Up 23 minutes (healthy)  
p11_pgadmin          Up 9 minutes  
p11_postgres         Up 23 minutes (healthy)  
[INFO] Nota: pgAdmin debe mostrar 'healthy' SOLO DESPUÉS de que postgres sea 'healthy'.  
  
Fase C - Detener contenedores y reiniciar (sin borrar volúmenes)  
¿Detener e reiniciar el stack ahora para validar persistencia? [s/N]: s  
[INFO] Deteniendo stack...  
WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion  
[+] down 6/6  
✔Container p11_nginx    Removed      0.4s  
✔Container p11_pgadmin  Removed      2.5s  
✔Container p11_app      Removed     10.3s  
✔Container p11_postgres Removed      0.3s  
✔Network red_publica    Removed      0.3s  
[INFO] Esperando 5 segundos...  
[INFO] Reiniciando stack...  
WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion  
[+] up 6/6  
✔Network red_publica    Created      0.1s  
✔Network red_datos      Created      0.1s
```

Figura 5.8 — Prueba 11.4 Fases A-C: Inserción de registro, healthchecks activos y docker compose down exitoso

```
aldair@srv-linux-sistemas: ~  
[INFO] Esperando 5 segundos...  
[INFO] Reiniciando stack...  
WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion  
[+] up 6/6  
✔Network red_publica    Created      0.1s  
✔Network red_datos      Created      0.1s  
✔Container p11_postgres  Healthy     12.0s  
✔Container p11_app       Started      1.4s  
✔Container p11_pgadmin   Started     12.3s  
✔Container p11_nginx     Started      1.9s  
[INFO] Esperando que postgres esté healthy (hasta 60 s)...  
  
Fase D - Verificar que los datos persisten  


| id | ts                     | msg                         |
|----|------------------------|-----------------------------|
| 1  | 2026-05-02 15:29:52+00 | Dato de prueba persistencia |

  
[OK] PRUEBA EXITOSA - Los datos sobrevivieron el reinicio del stack.  
El volumen practica11_postgres_data mantiene los datos persistentes.  
  
Estado final de healthchecks:  
WARN[0000] /home/aldair/practica-11/docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion  
NAME                STATUS  
p11_app              Up 11 seconds (health: starting)  
p11_nginx            Up 10 seconds (health: starting)  
p11_pgadmin          Up less than a second  
p11_postgres         Up 11 seconds (healthy)  
Presiona ENTER para continuar...
```

Figura 5.9 — Prueba 11.4 Fase D: Reinicio del stack; postgres healthy primero y dato persistente en el volumen practica11_postgres_data

6. Conclusiones y Referencias

6.1 Lecciones Aprendidas

Problema Encontrado	Solución Aplicada
docker-ce no inicia en Devuan (sysvinit)	Devuan usa sysvinit en lugar de systemd. Los scripts postinstall de docker-ce asumen systemd y fallan silenciosamente. La solución fue desinstalar docker-ce y usar docker.io del repositorio Debian, que incluye scripts init.d compatibles. El daemon se inicia con /etc/init.d/docker start.
PATH de root no incluye /sbin en Devuan	Al ejecutar dpkg --configure -a, los subprocessos como ldconfig fallaban con 'not found in PATH'. Se resolvió ejecutando export PATH=\$PATH:/usr/sbin:/sbin:/usr/local/sbin al inicio de sesión y agregándolo a /root/.bashrc para persistencia permanente.
pgAdmin rechaza emails con dominio .local	Las versiones recientes de pgAdmin4 validan el dominio del email con una librería de entregabilidad de correo. El dominio admin@practica11.local es rechazado por ser un dominio reservado. Se resolvió cambiando el email a admin@gmail.com en el archivo .env.
more_clear_headers no disponible en nginx:alpine	La directiva more_clear_headers requiere el módulo ngx_headers_more, no incluido en nginx:alpine. Nginx fallaba al arrancar con error de directiva desconocida. Se eliminó con sed -i y se sustituyó por proxy_hide_header Server para ocultar cabeceras.
Red internal:true bloquea el port binding	La opción internal: true bloquea todo tráfico externo incluyendo el binding del puerto al host. pgAdmin no podía exponer el puerto 8888 al localhost del servidor. Se cambió a internal: false manteniendo el aislamiento mediante la ausencia de puerto público en el Compose.
Permisos en /var/lib/pgadmin/sessions	El directorio montado /tmp/pgadmin_sessions tenía permisos incorrectos. pgAdmin usa el UID 5050 internamente. Se resolvió con rm -rf /tmp/pgadmin_sessions && mkdir -p /tmp/pgadmin_sessions && chown -R 5050:5050 /tmp/pgadmin_sessions.
SSH known_hosts cambia al reinstalar openssh	Al reinstalar openssh-server, el servidor regenera sus claves host. Windows rechaza la conexión con 'REMOTE HOST IDENTIFICATION HAS CHANGED'. Se resolvió ejecutando ssh-keygen -R 192.168.56.10 en PowerShell para limpiar la entrada antigua.
sudo no existe cuando se trabaja como root	En Devuan, al iniciar sesión directamente como root, sudo no existe en el PATH. Todos los comandos administrativos se ejecutan directamente sin prefijo. Los scripts del menú verifican EUID == 0 para validar privilegios sin depender de sudo.

6.2 Bibliografía

1. Docker Inc. (2025). Docker Engine — Instalación en Debian/Devuan. <https://docs.docker.com/engine/install/debian/>
2. Docker Inc. (2025). Docker Compose — Referencia del archivo docker-compose.yml (redes, volúmenes, healthcheck, depends_on). <https://docs.docker.com/compose/compose-file/>
3. Docker Inc. (2025). Networking en Docker: Bridge networks e internal networks. <https://docs.docker.com/network/bridge/>
4. nginx. (2025). nginx — Documentación de proxy_pass, upstream, server_tokens y proxy_hide_header. https://nginx.org/en/docs/http/nginx_http_proxy_module.html

5. PostgreSQL Global Development Group. (2025). PostgreSQL 15 — pg_isready y configuración de healthcheck en contenedores. <https://www.postgresql.org/docs/15/app-pg-isready.html>
6. pgAdmin Development Team. (2025). pgAdmin 4 — Despliegue en contenedores, variables de entorno y permisos de sesión. https://www.pgadmin.org/docs/pgadmin4/latest/container_deployment.html
7. OpenBSD Project. (2024). ssh(1) — Manual de referencia: Local Port Forwarding (opción -L). <https://man.openbsd.org/ssh>
8. Devuan Project. (2024). Devuan GNU+Linux — Filosofía init freedom, compatibilidad sysvinit. <https://devuan.org/>
9. UFW Community. (2024). UFW — Uncomplicated Firewall: políticas, reglas y configuración en Debian. <https://help.ubuntu.com/community/UFW>
10. Anthropic Claude. (claude.ai) — Se utilizaron prompts iterativos para diseñar los 5 scripts Bash de la Práctica 11: menu.sh (menú principal interactivo), lib/prerequisitos.sh (instalación de Docker y generación de archivos), lib/infraestructura.sh (gestión del stack Docker Compose), lib/firewall.sh (configuración UFW) y lib/pruebas.sh (protocolo de pruebas de aceptación). También se utilizó para resolución de problemas con Devuan sysvinit, docker-ce, pgAdmin, túneles SSH y permisos de contenedor. Modelo: Claude Sonnet 4.6. <https://claude.ai>